Operating System Final Exam (2025)

Name : **Student ID :**

1. Please explain what is **Race Condition** (5%) and **Critical Section**. (5%)

2. (a) Use an execution example to explain why the Program A allows two processes to enter the critical section simultaneously. (3%) (b) Use an execution example to explain why the Program B allows two processes to enter the critical section simultaneously. (3%) (c) Write the correct program. (4%)

```
// Program A (Process i)

while (true) {
    turn = j;
    flag[i] = true;
    while (flag[j] && turn == j)
        ;
    // critical section
    flag[i] = false;
}
```

```
// Program B (Process i)

while (true) {
    flag[i] = true;
    turn = i;
    while (flag[j] && turn == j)
        ;
    // critical section
    flag[i] = false;
}
```

3. A semaphore ***sem*** can be used to implement a critical section as follows:

```
wait(sem);
/* critical section */
signal(sem);
```

(a) What should be the initial value of ***sem*** in order for this code to correctly implement mutual exclusion? (1%)
(b) Explain what will happen if the initial value of ***sem*** is set to 0? (4%)

A semaphore ***sync*** can also be used to enforce an execution order between two statements. For example, to ensure that S2 executes only after S1, we can write:

```
// Process 1
S1;
signal(sync);

// Process 2
wait(sync);
S2;
```

(a) What should be the initial value of ***sync***? (1%)
(b) Explain what will happen if the initial value of ***sync*** is set to 1? (4%)

4. (a) Explain the differences in the **waiting mechanisms** used by **Spinlocks** and **Semaphores** in their implementations. (5%) (b) Describe the *advantages* of each of these two waiting mechanisms. (5%)

5. Please explain the four necessary conditions for a deadlock to occur. (4%) Given the following code:

```
void do_work_one() {
    first_mutex.lock();
    second_mutex.lock();
    std::cout << "do_work_one" << std::endl;
    second_mutex.unlock();
    first_mutex.unlock();
}

void do_work_two() {
    second_mutex.lock();
    first_mutex.lock();
    std::cout << "do_work_two" << std::endl;
    first_mutex.unlock();
    second_mutex.unlock();
}
```

Process 1 executes do_work_one(), and Process 2 executes do_work_two().

Please provide two different ways to rewrite this program so that deadlock does not occur. For each rewritten version, you must explain which deadlock condition is no longer satisfied. (6%)

6. Please explain what is **External Fragmentation** (5%), and what is **Internal Fragmentation** (5%).

7. Please explain the pros and cons of having too large or too small a page size. (10%)

8. Please describe two solutions for handling a page table that is too large to fit into a single frame. A detailed explanation is required. (10%)

9. Given five memory partitions of 180 Kb, 500 Kb, 260 Kb, 300 Kb, 620 Kb (in order), how would the **first-fit** (3%), **best-fit** (3%), and **worst-fit** (4%) algorithms place processes of 230 Kb, 390 Kb, 150 Kb, and 450 Kb (in order).

10. Assume that we have 3 frames, and the memory reference string is 6, 1, 2, 0, 2, 3, 2, 5, 1, 0, 3, 2, 3, 0, 2, 0, 1, 6, 0, 1. Write the results of **FIFO** page replacement (3%), **Optimal** page replacement (3%) and **LRU** page replacement (4%). You should count how many times page fault occurs.